

Morden Application Development Project 1

Trekking Management Application

1. Student Details

Name: Srujan Rana

Roll Number: 24F1001070

Email: 24f1001070@ds.study.iitm.ac.in

About Me: I am a final-year B.Tech. student at Sambalpur University and am also pursuing a dual degree in Data Science through the BS Degree Programme at IIT Madras. I have a strong interest in web application development. I enjoy building meaningful applications that combine software development, data management, and user experience.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Adventure organizations often manage trekking activities using spreadsheets, phone calls, messaging applications, and other manual methods. Managing trek details, staff assignments, participant registrations, and bookings through these methods can become difficult as the number of treks and participants increases.

The absence of a centralized management system can lead to problems such as **overbooking, difficulty in tracking available slots, inefficient staff coordination, and difficulty maintaining booking records.**

The objective of this project is to develop a web-based **Trekking Management Application** that provides a centralized platform for managing trekking activities. The application provides different functionalities based on three user roles: **Admin, Staff, and User.**

The Admin is responsible for managing treks, approving staff members, assigning staff to treks, and monitoring users and bookings. Staff members can manage the treks assigned to them, update available slots and trek status, and view registered participants. Users can browse available treks, book a trek, and view their booking history.

The application also implements authentication, role-based access control, password hashing, session management, duplicate booking prevention, and overbooking prevention.

Approach:

The application was developed using **Flask** as the backend framework with a modular architecture using Flask Blueprints. **SQLite** is used for persistent data storage through **Flask-SQLAlchemy**.

The application implements authentication and role-based access control using **Flask-Login**. Separate dashboards are provided for Admin, Staff, and Users.

3. AI/LLM Declaration

I used **Perplexity** primarily for research, studying relevant concepts, and exploring different approaches to develop a better and more effective application. It was used to understand application development practices, Flask concepts, and possible implementation approaches.

I used **ChatGPT (GPT-5.6 Luna)** mainly during the implementation phase, particularly for developing and improving the **HTML and Jinja2 templates**, structuring frontend components, and **debugging errors** encountered during development.

The extent of AI/LLM usage is around **15–18%**, limited to **code suggestions and concept understanding**.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Python	Core programming language

Flask	Backend web application framework
Flask-SQLAlchemy	Object Relational Mapper for SQLite
SQLite	Database for storing application data
Flask-Login	Authentication and session management
Werkzeug	Password hashing and password verification
Jinja2	Dynamic HTML template rendering
HTML5	Structure of web pages
CSS	Custom styling
Bootstrap 5	Responsive UI and component styling

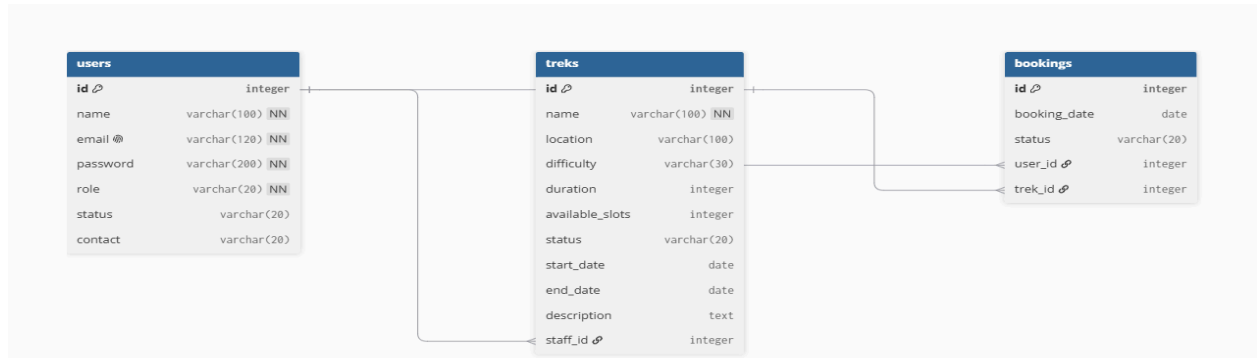
5. Database Schema / ER Diagram

Tables:

1. **Users** : Stores information about Admin, Staff, and regular Users. (id , name, email,password, role, status, contact)
2. **Treks** : Stores information about trekking activities.(id, name, location, difficulty, duration, available_slots, status, start_date, end_date, description, staff_id)
3. **Bookings**: Stores trek booking information. (id, booking_date, status, user_id, trek_id)

Relations:

One User → Many Bookings
One Trek → Many Bookings
One Staff User → Many Treks



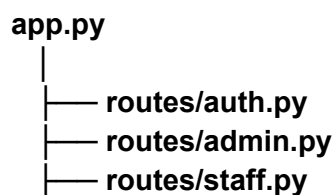
6. API Resource Endpoints

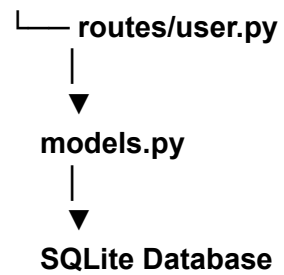
Endpoint	Method	Description
/login	GET, POST	Authenticate user
/register	GET, POST	Register User/Staff
/logout	GET	Logout current user
/admin/dashboard	GET	Admin dashboard
/admin/treks	GET	View all treks
/admin/add-trek	GET, POST	Create a trek
/admin/edit-trek/<id>	GET, POST	Edit a trek
/admin/delete-trek/<id>	GET	Delete a trek
/admin/approve-staff	GET	View pending staff
/admin/approve-staff/<id>	GET	Approve staff

/admin/assign-staff/<id>	GET, POST	Assign staff to trek
/admin/users	GET	View users and staff
/admin/bookings	GET	View all bookings
/admin/search	GET	Search users, staff and treks
/staff/dashboard	GET	Staff dashboard
/staff/treks	GET	View assigned treks
/staff/manage-trek/<id>	GET, POST	Update slots/status
/staff/participants/<id>	GET	View trek participants
/user/dashboard	GET	User dashboard
/user/treks	GET	View available treks
/user/book/<id>	GET	Book a trek
/user/my-bookings	GET	View booking history

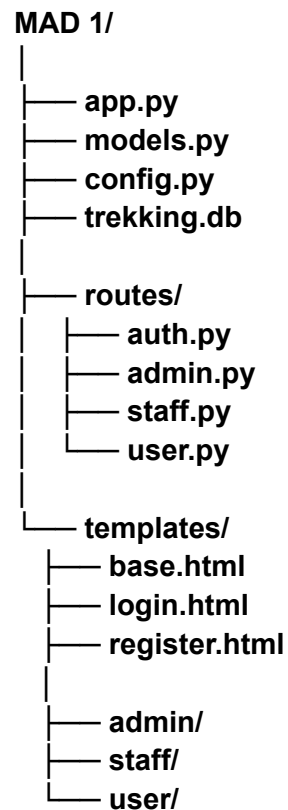
7. Architecture

The application follows a modular Flask architecture.





Project Structure



The Video for the project

 MAD-1 Project Video.mp4